

react

声明式: 直接在组件声明

命令式: 给属性赋值

```
npm create vite@latest my-react-app -- --template react-ts
```

对比其他

vue 渐进式框架, 框架

react 视图库, JSX

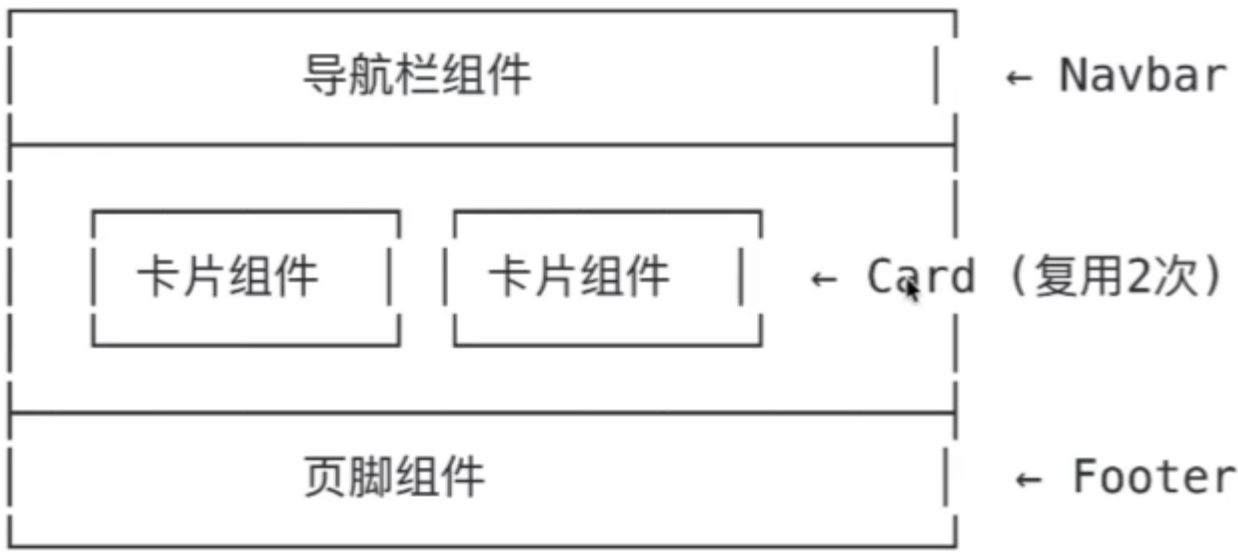
使用react原因:

- 1. hooks 管理状态简单
- 2. 可同时做手机和网页
- 3. 丰富成熟

介绍

node_modules 组件位置

页面结构示例:



步骤1: 定义 Props 类型

```
1 interface GreetingProps {  
2   name: string;           // 必填属性  
3   age?: number;           // 可选属性 (?)  
4 }
```

步骤2: 组件接收 Props (解构写法)

```
1 function Greeting({ name, age }: GreetingProps) {  
2   // 解构赋值 = const { name, age } = props  
3   return (  
4     <div>  
5       <h1>你好, {name}! </h1>  
6       { /* 条件渲染: 有 age 才显示 */ }  
7       {age && <p>{age}岁</p>}  
8     </div>  
9   );  
10 }
```

步骤3: 使用组件 (父传子)

```
1 // 方式1: 单独传属性  
2 <Greeting name="小明" age={20} />  
3  
4 // 方式2: 展开传对象 (推荐数据来自变量时)  
5 const user = { name: "小红", age: 18 };  
6 <Greeting {...user} />  
7 // 等价于 <Greeting name="小红" age={18} />
```

🗨️ 互动问答 3：数据流向

问题：看下面的代码，数据是从哪里流向哪里？

```
1 function App() { // A
2   return <Button text="点击" />; // B
3 }
4
5 function Button({ text }: { text: string }) { // C
6   return <button>{text}</button>; // D
7 }
```

使用 TypeScript 描述出来

子要在上

```
function Button({age}){}
function App(){
  return <Button age=""><button/>
}
```

子组件通知父组件

答案：父组件传一个回调函数，子组件调用它来通知父组件

```
1 interface ButtonProps {
2   text: string;
3   onClick: () => void; // 父组件传下来的回调
4 }
5
6 function Button({ text, onClick }: ButtonProps) {
7   // 子组件调用，但控制权在父组件
8   return <button onClick={onClick}>{text}</button>;
9 }
10
11 // 父组件使用
12 <Button text="点击我" onClick={() => console.log('被点击了')} />
```

1

1

```

7 // 按钮 Props 继承基础 Props
8 interface ButtonProps extends BaseProps {
9     variant: "primary" | "secondary";
10    onClick: () => void;
11 }
12
13 // 链接 Props 也继承
14 interface LinkProps extends BaseProps {
15     href: string;
16 }
17
18 // 方式2: 函数式更新 (新值依赖旧值, 更安全)
19 setCount(prev => prev + 1);

```

这是新值依赖旧值

💡 对象 State 的更新

```
1 interface User {  
2   name: string;  
3   age: number;  
4 }  
5  
6 const [user, setUser] = useState<User>({  
7   name: '',  
8   age: 0  
9 });
```

✅ ** 正确：展开运算符创建新对象 **

```
1 const updateName = (newName: string) => {  
2   setUser({ ...user, name: newName });  
3   // 保留原有 age, 只覆盖 name  
4 };
```

❌ ** 错误：直接修改不会触发更新 **

```
1 user.name = 'Tom'; // 修改了原对象  
2 setUser(user);     // 同一引用, React 认为没变
```

应该使用 `[]` 或 `{}` 创建权限数组或对象再 `set` 相关()

⚠️ 嵌套对象的浅拷贝陷阱

```
1 // 深度嵌套需要逐层展开  
2 setState(prev => ({  
3   ...prev,  
4   user: {  
5     ...prev.user,  
6     tags: [...prev.user.tags, '新标签'] // 新数组  
7   }  
8 }));
```

依次定义内部新的

事件	类型	获取值
onClick	<code>React.MouseEvent<HTMLElement></code>	<code>e.currentTarget</code>
onChange	<code>React.ChangeEvent<HTMLInputElement></code>	<code>e.target.value</code>
onSubmit	<code>React.FormEvent<HTMLFormElement></code>	<code>e.preventDefault()</code>
onKeyDown	<code>React.KeyboardEvent<HTMLInputElement></code>	<code>e.key</code>

取值获取值即可

受控组件模式

```
1  const [value, setValue] = useState('');
2
3  <input
4    value={value}           // React 控制值
5    onChange={e => setValue(e.target.value)} // React 响应变化
6  />
```

优点：

- 数据变化可追踪
- 可以做实时校验
- 多个输入可以统一管理

受控组件: 根据事件获取组件的值

节流: 均匀取

防抖: 取最后一次

事件频繁触发导致性能下降

```
1 interface Item {  
2   id: string;      // 必须有唯一 key  
3   name: string;  
4 }  
5  
6 function List({ items }: { items: Item[] }) {  
7   return (  
8     <ul>  
9       {items.map(item => (  
10        <li key={item.id}>{item.name}</li>  
11        ))}  
12     </ul>  
13   );  
14 }
```

li 设置 key

根据idx时, idx不消失, 值就不会改

不允许中文做key 可能有编码问题

若id不唯一, 拼接得到唯一

时间戳有时间问题, 时间差会过小